

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 1 – מעודכן לתאריך 21.05.2026
עמוד 1 מתוך 9



מתרגל ממונה על התרגיל: איל רביד, eyal.ravid@campus.technion.ac.il

תאריך ושעת הגשה: 14/6/2026 בשעה 23:59

אופן ההגשה: בזוגות. אין להגיש ביחידים. (אלא באישור מתרגל אחראי של הקורס).

הנחיות כלליות:

- שאלות על התרגיל יש לפרסם באתר הפיאצה של הקורס תחת לשונית "wet1":
 - האתר: <https://piazza.com/technion.ac.il/spring2026/234218>, נא לקרוא את השאלות של סטודנטים אחרים לפני שמפרסמים שאלה חדשה, למקרה שנשאלה כבר.
- נא לקרוא את המסמך "נהלי הקורס" באתר הקורס. בנוסף, נא לקרוא בעיון את כל ההנחיות בסוף מסמך זה.
- בפורום הפיאצה ינוהל FAQ ובמידת הצורך יועלו תיקונים כהודעות נעוצות (Pinned Notes). תיקונים אלו

מחייבים.

- התרגיל מורכב משני חלקים: יבש ורטוב.
 - לאחר קריאת כלל הדרישות, מומלץ לתכנן תחילה את מבני הנתונים על נייר. דבר זה יכול לחסוך לכם זמן רב.
 - לפני שאתם ניגשים לקודד את פתרוןכם, ודאו כי יש לכם פתרון העומד בכל דרישות הסיבוכיות בתרגיל. תרגיל שאינו עומד בדרישות הסיבוכיות יחשב כפסול.
 - את הפתרון שלכם מומלץ לחלק למחלקות שונות שאפשר לממש (ולבדוק!) בהדרגתיות.
 - המלצות לפתרון התרגיל נמצאות באתר הקורס תחת: "Programming Tips Session".
- המלצות לתכנות במסמך זה אינן מחייבות, אך מומלץ להיעזר בהן.
- חומר התרגיל הינו כל החומר שנלמד בהרצאות ובתרגולים עד אך לא כולל עצי דרגות.
- העתקת תרגילי בית רטובים תיבדק באמצעות תוכנת בדיקות אוטומטית, המזהה דמיון בין כל העבודות הקיימות במערכת, גם כאלו משנים קודמות. לא ניתן לערער על החלטת התוכנה. התוכנה אינה מבדילה בין מקור להעתק! אנא הימנעו מהסתכלות בקוד שאינו שלכם.
- בקשות להגשה מאוחרת יש להפנות למתרגל האחראי בלבד בכתובת: goldshtein@campus.technion.ac.il

מבני נתונים 234218 אביב תשפ"ו

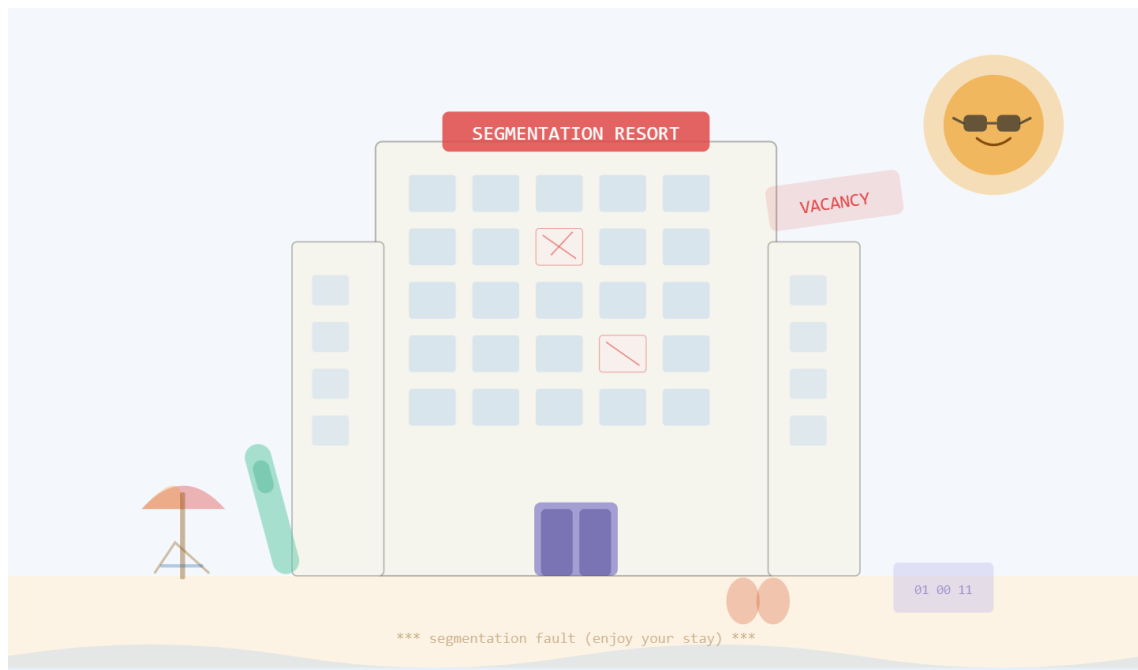
גיליון רטוב מספר 1 – מעודכן לתאריך 21.05.2026

עמוד 2 מתוך 9



הקדמה:

כבכל קיץ, עם תום הסמסטר בניין טאוב מתרוקן. הסטודנטים בחופש, הסגל בכנסים בחו"ל, והמזגנים עובדים לשווא. לאור מצב זה, החליטה הנהלת הפקולטה להסב את בניין טאוב למלון היוקרה "Segmentation Resort". משרדי הפקולטה יוסבו לחדרי אירוח, הספרייה תשמש כחדר אוכל, והחווה תהפוך לבר קוקטיילים. הפקולטה פונה בזאת לסטודנטים החרוצים בקורס לבנות את מערכת ניהול המלון, בהתנדבות כמובן, שהרי ניסיון הוא התשלום הטוב ביותר. כמו כן, לאור המצב הקשה בשוק ההייטק, הוחלט כי סטודנטי הפקולטה ישמשו גם כצוות הניקיון של המלון. ניסיון נוסף לקורות החיים.



Segmentation Resort

חופשה שמסתיימת ב-fault

Taub Building, Technion Campus



סימונים לצורכי סיבוכיות:

נסמן ב- n את מספר האורחים במלון.

ב- m את מספר השולחנות הפעילים בחדר האוכל.

ב- $n_{tableId}$ את מספר האורחים היושבים בשולחן בעל מזהה $tableId$ אם אין שולחן כזה אפשר להתייחס לערך זה כ-1.

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 1 – מעודכן לתאריך 21.05.2026
עמוד 3 מתוך 9



דרוש מבנה נתונים למימוש הפעולות הבאות:

SegmentationResort()

מאתחלת מבנה נתונים ריק. תחילה אין במערכת אורחים או שולחנות פעילים.

פרמטרים: אין

ערך החזרה: אין

סיבוכיות זמן: $O(1)$ במקרה הגרוע.

virtual ~SegmentationResort()

הפעולה משחררת את המבנה (כל הזיכרון אותו הקצאתם חייב להיות משוחרר).

פרמטרים: אין

ערך החזרה: אין

סיבוכיות זמן: $O(n + m)$ במקרה הגרוע.

StatusType checkIn(int guestId, int roomNum)

אורח בעל מזהה guestId נכנס לחדר מספר roomNum.

פרמטרים:

guestId מזהה האורח שצריך להוסיף.

roomNum מספר החדר בו האורח ישהה.

ערך החזרה:

ALLOCATION_ERROR במקרה של בעיה בהקצאה/שחרור זיכרון.

INVALID_INPUT אם $guestId \leq 0$ או $roomNum \leq 0$.

FAILURE אם קיים כבר במערכת אורח עם מזהה guestId או שחדר מספר roomNum

כבר תפוס.

SUCCESS במקרה של הצלחה.

סיבוכיות זמן: $O(\log n)$ במקרה הגרוע.

StatusType checkOut(int guestId)

אורח בעל המזהה guestId עוזב את המלון ולכן צריך להוציאו מהמערכת. אם האורח נמצא כרגע בחדר האוכל אז הוא לא יכול לעשות checkOut והפעולה תכשל.

פרמטרים:

guestId מזהה האורח.

ערך החזרה:

ALLOCATION_ERROR במקרה של בעיה בהקצאה/שחרור זיכרון.

INVALID_INPUT אם $guestId \leq 0$.

FAILURE אם אין אורח עם מזהה guestID במערכת או שהאורח נמצא בחדר אוכל.

SUCCESS במקרה של הצלחה.

סיבוכיות זמן: $O(\log n)$ במקרה הגרוע.

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 1 – מעודכן לתאריך 21.05.2026
עמוד 4 מתוך 9



StatusType addTable(int tableId, int capacity)

מוסיפים שולחן בעל מזהה ייחודי tableId לחדר האוכל, מספר הכיסאות בשולחן הוא capacity וזה גם מספר האורחים המקסימלי שיכולים לשבת סביב שולחן זה.

פרמטרים:

מזהה השולחן שנוסף למערכת.	tableId
מספר האורחים המקסימלי שיכולים לשבת סביב שולחן זה.	capacity

ערך החזרה:

במקרה של בעיה בהקצאה/שחרור זיכרון.	ALLOCATION_ERROR
אם $tableId \leq 0$ או אם $capacity \leq 0$.	INVALID_INPUT
אם קיים כבר שולחן בעל מזהה tableId במערכת.	FAILURE
במקרה של הצלחה.	SUCCESS

סיבוכיות זמן: $O(\log m)$ במקרה הגרוע.

StatusType removeTable(int tableId)

מסירים את השולחן בעל המזהה tableId מהמערכת. אם יש אורחים שיושבים סביב השולחן הפעולה תכשל והשולחן לא יוסר.

פרמטרים:

מזהה השולחן שמבקשים להסיר.	tableId
----------------------------	---------

ערך החזרה:

במקרה של בעיה בהקצאה/שחרור זיכרון.	ALLOCATION_ERROR
אם $tableId \leq 0$.	INVALID_INPUT
אם אין שולחן עם מזהה tableId או שיש אורחים שיושבים סביב שולחן זה.	FAILURE
במקרה של הצלחה.	SUCCESS

סיבוכיות זמן: $O(\log m)$ במקרה הגרוע.

StatusType enterDiningRoom(int guestId, int tableId)

אורח בעל המזהה guestId מבקש להיכנס לחדר האוכל ולהתיישב בשולחן tableId. אם האורח כבר נכח בארוחה הנוכחית (גם אם יצא מחדר האוכל), הכניסה תידחה והוא יוכל להיכנס רק בארוחה הבאה.

פרמטרים:

מזהה האורח שנכנס לחדר האוכל.	guestId
השולחן בו האורח מבקש לשבת.	tableId

ערך החזרה:

במקרה של בעיה בהקצאה/שחרור זיכרון.	ALLOCATION_ERROR
אם $tableId \leq 0$ או אם $guestId \leq 0$.	INVALID_INPUT
אם אין אורח עם מזהה guestId או אין שולחן עם מזהה tableId במערכת או שהאורח כבר נמצא בחדר האוכל או שהשולחן מלא או שהאורח כבר היה בחדר אוכל בארוחה הנוכחית.	FAILURE
במקרה של הצלחה.	SUCCESS

סיבוכיות זמן: $O(\log n + \log m)$ במקרה הגרוע.

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 1 – מעודכן לתאריך 21.05.2026
עמוד 5 מתוך 9



`StatusType leaveDiningRoom(int guestId, int tableId)`

אורח בעל המזהה `guestId` עוזב את השולחן בעל המזהה `tableId` ויוצא מחדר האוכל.

פרמטרים:

<code>guestId</code>	מזהה האורח שנכנס לחדר האוכל.
<code>tableId</code>	השולחן בו האורח היה.

ערך החזרה:

ALLOCATION_ERROR	במקרה של בעיה בהקצאה/שחרור זיכרון.
INVALID_INPUT	אם <code>tableId <= 0</code> או אם <code>guestId <= 0</code>
FAILURE	אם אין אורח עם מזהה <code>guestId</code> או אין שולחן עם מזהה <code>tableId</code> במערכת או שהאורח לא יושב בשולחן בעל המזהה <code>tableId</code> .
SUCCESS	במקרה של הצלחה.

סיבוכיות זמן: $O(\log m + \log n_{tableId})$ במקרה הגרוע.

`StatusType reheatFood()`

ארוחה חדשה מתחילה בחדר האוכל. אורחים שהשתתפו בארוחות קודמות רשאים להיכנס מחדש. אורחים שנותרו בחדר האוכל מהארוחה הקודמת נחשבים כמשתתפים בארוחה החדשה, ולכן אם יצאו במהלך הארוחה החדשה, הם יוכלו לחזור אליה. מאחר שמדובר בחימום מחדש של האוכל מהארוחה הקודמת ללא הכנה של אוכל חדש, סיבוכיות הזמן של הפעולה צריכה להיות $O(1)$.

ערך החזרה:

ALLOCATION_ERROR	במקרה של בעיה בהקצאה/שחרור זיכרון.
SUCCESS	במקרה של הצלחה.

סיבוכיות זמן: $O(1)$ במקרה הגרוע.

`StatusType joinTables(int tableId1, int tableId2)`

הסועדים בחדר האוכל מבקשים לאחד את השולחנות בעלי המזהים `tableId1` ו `tableId2` המזהה של השולחן המאוחד יהיה `tableId1` ומספר המקומות (`capacity`) בשולחן המאוחד יהיה כסכום מספרי המקומות של השולחנות המקוריים. היושבים בשולחנות המקוריים עוברים לשבת ביחד בשולחן המאוחד.

פרמטרים:

<code>tableId1</code>	מזהה השולחן הראשון שמאחדים.
<code>tableId2</code>	מזהה השולחן השני שמאחדים.

ערך החזרה:

ALLOCATION_ERROR	במקרה של בעיה בהקצאה/שחרור זיכרון.
INVALID_INPUT	אם <code>tableId2 <= 0</code> או אם <code>tableId1 <= 0</code>
FAILURE	אם אחד השולחנות לא קיימים במערכת.
SUCCESS	במקרה של הצלחה.

סיבוכיות זמן: $O(\log m + n_{tableId1} + n_{tableId2})$ במקרה הגרוע.

`output_t < int > joinFriend(int guestId1, int guestId2)`

אורח בעל מזהה `guestId1` רוצה להתיישב בחדר האוכל בשולחן של חברו בעל המזהה `guestId2`. האורח `guestId2` כבר נמצא בחדר האוכל והאורח `guestId1` רוצה להתיישב בשולחן שלו.

פרמטרים:

<code>guestId1</code>	מזהה האורח שרוצה להיכנס לחדר האוכל.
<code>guestId2</code>	מזהה האורח שבשולחנו רוצה להתיישב האורח בעל המזהה <code>guestId1</code> .

ערך החזרה:

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 1 – מעודכן לתאריך 21.05.2026

עמוד 6 מתוך 9



במקרה של בעיה בהקצאה/שחרור זיכרון.	ALLOCATION_ERROR
אם $guestId1 \leq 0$ או אם $guestId2 \leq 0$	INVALID_INPUT
או אם $guestId1 == guestId2$	
אם אחד מהאורחים לא קיים במערכת או ש $guestId2$ לא נמצא בחדר	FAILURE
האוכל או שהשולחן שבו יושב $guestId2$ בתפוסה מלאה או אם $guestId1$	
כבר נכח בחדר האוכל בארוחה הנוכחית.	
במקרה של הצלחה יוחזר גם מזהה השולחן שבו יושבים האורחים.	SUCCESS
סיבוכיות זמן: $O(\log n)$ במקרה הגרוע.	

`output_t < int > cleanNextRoom()`

צוות הניקיון מנקה את החדר התפוס הבא במלון. החדרים מנוקים לפי הסדר הבא:

- בכל קריאה לפונקציה, הצוות עובר לחדר התפוס בעל מזהה החדר המינימלי שגדול ממזהה החדר האחרון שנוקה.
- אם לא קיים חדר תפוס כזה (כלומר, כל החדרים התפוסים הם בעלי מזהה קטן או שווה למזהה החדר האחרון שנוקה), הצוות חוזר לחדר התפוס בעל המזהה המינימלי במלון.
- בקריאה הראשונה לפונקציה (לפני שנוקה אי פעם חדר), הצוות מתחיל מהחדר התפוס בעל המזהה המינימלי.

ערך החזרה:

במקרה של בעיה בהקצאה/שחרור זיכרון.	ALLOCATION_ERROR
אם אין אורחים במלון.	FAILURE
במקרה של הצלחה יוחזר גם מזהה החדר שאליו הגיעו צוות הניקיון.	SUCCESS
סיבוכיות זמן: $O(1)$ במקרה הגרוע.	



דוגמה הרצה:

```
checkIn(2, 4): SUCCESS
checkIn (1, 10): SUCCESS
checkIn (5, 5): SUCCESS
addTable(10, 2): SUCCESS
addTable (11, 1): SUCCESS
enterDiningRoom(2, 10): SUCCESS
joinFriend(1, 2): SUCCESS , Value: 10
joinFriend(1, 2): FAILURE
joinFriend(5, 2): FAILURE
joinTables(11, 10): SUCCESS
joinFriend(5, 2): SUCCESS , Value: 11
leaveDiningRoom(2, 11): SUCCESS
enterDiningRoom(2, 11): FAILURE
reheatFood(): SUCCESS
enterDiningRoom(2, 11): SUCCESS
checkOut(2): FAILURE
leaveDiningRoom(2, 11): SUCCESS
checkOut(2): SUCCESS
```

סיבוכיות מקום:

סיבוכיות המקום הדרושה עבור מבנה הנתונים היא $O(n + m)$ במקרה הגרוע. כלומר בכל רגע בזמן הריצה, צריכת המקום של מבנה הנתונים תהיה לינארית בסכום מספרי האורחים, השולחנות במערכת. סיבוכיות המקום הנדרשת עבור כל פעולה (כלומר, זיכרון "העזר" שכל פעולה משתמשת בו) אינה מצוינת לכל פעולה לחד, אך אסור לעבור את סיבוכיות המקום הדרושה שמוגדרת לכל המבנה.

ערכי החזרה של הפונקציות:

כל אחת מהפונקציות מחזירה ערך מטיפוס `StatusType` שייקבע לפי הכלל הבא:

- תחילה, יוחזר `INVALID_INPUT` אם הקלט אינו תקין.
 - אם לא הוחזר `INVALID_INPUT`:
 - בכל שלב בפונקציה, אם קרתה שגיאת הקצאה/שחרור יש להחזיר `ALLOCATION_ERROR`. מצב זה אינו צפוי אלא באחד משני מקרים (לרוב): באמת השתמשתם בקלט גדול מאוד ולכן המבנה ניצל את כל הזיכרון במערכת, או שיש זליגת זיכרון בקוד.
 - אם קרתה שגיאה אחרת, כפי שמצוין בכל פונקציה, יש להחזיר מיד `FAILURE` מבלי לשנות את מבנה הנתונים.
 - אחרת, יוחזר `SUCCESS`.
- חלק מהפונקציות צריכות להחזיר בנוסף עוד פרמטר (`bool` או `int`), לכן הן מחזירות אובייקט מטיפוס `output_t<T>`. אובייקט זה מכיל שני שדות: הסטטוס (`__status`) ושדה נוסף (`__ans`) מסוג `T`.



במקרה של הצלחה (SUCCESS), השדה הנוסף יכיל את ערך החזרה, והסטטוס יכיל את SUCCESS. בכל מקרה אחר, הסטטוס יכיל את סוג השגיאה והשדה הנוסף לא מעניין.

שני הטיפוסים (output_t<T>, StatusType) ממומשים כבר בקובץ "wet1util.h" שניתן לכם כחלק מהתרגיל.

קיים קונסטרקטור של output_t<T> מ-T ומ-StatusType כך שניתן פשוט לכתוב בפונקציות הרלוונטיות:

```
return 7;
```

```
return StatusType::FAILURE;
```

הנחיות ודגשים כלליים:

חלק יבש:

- החלק היבש מהווה חלק מהציון על התרגיל כפי שמצוין בנהלי הקורס.
- לפני מימוש הפעולות בקוד יש לתכנן היטב את מבני הנתונים והאלגוריתמים ולוודא כי באפשרותכם לממש את הפעולות בדרישות הזמן והזיכרון שלעיל.
- החלק היבש חייב להיות מוקלד.
- הגשת החלק הרטוב מהווה תנאי הכרחי לקבלת ציון על החלק היבש, כלומר, הגשה בה יתקבל אך ורק חלק יבש תגרור ציון 0 על התרגיל כולו.
- יש להכין מסמך הכולל תיאור של מבני הנתונים והאלגוריתמים בהם השתמשתם בצירוף הוכחת סיבוכיות הזמן והמקום שלהם. חלק זה עומד בפני עצמו וצריך להיות מובן לקורא גם לפני העיון בקוד. אין צורך לתאר את הקוד ברמת המשתנים, הפונקציות והמחלקות, אלא ברמה העקרונית. חלק יבש זה לא תיעוד קוד.
- ראשית הציגו את מבני הנתונים בהם השתמשתם. רצוי ומומלץ להיעזר בצירוף.
- לאחר מכן הסבירו כיצד מימשתם כל אחת מהפעולות הנדרשות. הוכיחו את דרישות סיבוכיות הזמן של כל פעולה תוך כדי התייחסות לשינויים שהפעולות גורמות במבני הנתונים.
- הוכיחו שמבנה הנתונים וכל הפעולות עומדים בדרישת סיבוכיות המקום.
- החסמים הנתונים בתרגיל הם לא בהכרח הדוקים ולכן יכול להיות שקיים פתרון בסיבוכיות טובה יותר. מספיק להוכיח את החסמים הדרושים בתרגיל.
- רמת פירוט: יש להסביר את כל הפרטים שאינם טריוויאליים ושחשובים לצורך מימוש הפעולות ועמידה בדרישות הסיבוכיות. אין לדון בפרטים טריוויאליים (הפעילו את שיקול דעתכם בקשר לזה, ושאלו את האחראי על התרגיל אם אינכם בטוחים). אין לצטט קטעים מהקוד כתחליף להסבר. אין צורך לפרט אלגוריתמים שנלמדו בכתה. כמו כן, אין צורך להוכיח תוצאות ידועות שנלמדו בכתה, אלא מספיק לציין בבירור לאיזו תוצאה אתם מתכוונים. אין (וגם אין צורך) להשתמש בתוצאות של עצי דרגות והלאה.
- **על חלק זה לא לחרוג מ-8 עמודים.**
- והכי חשוב **!keep it simple**

חלק רטוב:

- מומלץ לממש תחילה את מבני הנתונים בצורה הכללית ביותר ורק אז לממש את הפונקציות הנדרשות בתרגיל.
- אנו ממליצים בחום על מימוש **Object Oriented**, **C++**, מימוש כזה יאפשר לכם להגיע לפתרון פשוט וקצר יותר לפונקציות אותן עליכם לממש ויאפשר לכם להכליל בקלות את מבני הנתונים שלכם (זכרו שיש תרגיל רטוב נוסף בהמשך הסמסטר).
- פקודת הקימפול שמורצת ב-gradescope הינה: `g++ -std=c++14 -DNDEBUG -Wall -o main.out *.cpp`
- חתימות הפונקציות שעליכם לממש ומספר הגדרות נמצאים בקובץ `SegmentationResort26b1.h`.
- אין לשנות את הקבצים `main26b1.cpp` ו-`wet1util.h` אשר סופקו כחלק מהתרגיל, ואין להגיש אותם. ישנה בדיקה אוטומטית שאין בקוד שימוש ב-STL, ובדיקה זו נופלת אם מגישים גם את `main26b1.cpp`.
- את שאר הקבצים ניתן לשנות, ותוכלו להוסיף קבצים נוספים כרצונכם, ולהגיש אותם.
- העיקר הוא שהקוד שאתם מגישים יתקמפל עם הפקודה לעיל, כאשר מוסיפים לו את שני הקבצים `wet1util.h` ו-`main26b1.cpp`.
- עליכם לממש בעצמכם את כל מבני הנתונים (למשל אין להשתמש במבנים של STL ואין להוריד מבני נתונים מהאינטרנט). **כחלק מתהליך הבדיקה אנו נבצע בדיקה ידנית של הקוד ונוודא שאכן מימשתם את מבני הנתונים שבהם השתמשתם.**
- בפרט, אסור להשתמש ב-`std::pair`, `std::vector`, `std::iterator`, או כל אלגוריתם של STL, רשימה מלאה של הספריות להן אסור לעשות include נמצאת בקובץ `dont_include.txt`.
- **ניתן** להשתמש במצביעים חכמים (Smart pointers כמו `shared_ptr`), בספריית `math` או בספריית `exception`.
- חשוב לוודא שאתם מקצים/משחררים זיכרון בצורה נכונה (מומלץ לוודא עם `valgrind`). לא חייבים לעבוד עם מצביעים חכמים, אך אם אתם מחליטים כן לעשות זאת, לוודא שאתם משתמשים בהם נכון. (תזכרו שהם לא פתרון



- קסם, למשל, כאשר יוצרים מעגל בהצבעות). שימו לב שהעתקת מצביעים חכמים היא איטית מאד ולכן יכולה לגרום ל-timeout. עדיף להעביר shared_ptr כשצריכים by reference.
- שגיאות של ALLOCATION_ERROR בד"כ מעידות על זליגה בזיכרון.
- על הקוד להתקמפל ולעבור את כל הבדיקות שמפורסמות לכם ב-gradescope. הטסטים שמורצים באתר מייצגים את הבדיקת אותן נריץ בנתינת הציון, כאשר פרסמנו 5 מתוך 50.
- אותם טסטים שבgradescope גם מפורסמים כקבצי קלט ופלט, יחד עם סקריפט בשם run_tests.py שנכתב בשביל python 3.6 ומעלה, המאפשר לבדוק את הקוד שלכם. מומלץ לבדוק את התרגיל לוקאלית לפני שמגישים. במידה ויש timeout על אחד מהטסטים זה יחשב ככשלון בטסט. עליכם לכתוב קוד יעיל במידת הסביר. אין לדאוג מכך יותר מדי, הרוב המוחלט של הפתרונות שעומד בסיבוכיות עומד גם בזמני הריצה.
- שימו לב:** התוכנית שלכם תיבדק על קלטים רבים ושונים מקבצי הדוגמא הנ"ל. יחד עם זאת הטסטים האלו מייצגים מבחינת אורך ואופן היצירה שלהם את השאר.

אופן ההגשה:

הגשת התרגיל מתבצעת דרך אתר ה- gradescope של הקורס.

חלק הרטוב:

יש להגיש רק את קבצי הקוד שלכם (לרוב קבצי .h, .cpp, בלבד אפשר גם להגיש אותם כקובץ zip) ב- gradescope תחת המשימה "wet 1 code part".

חלק היבש:

יש להגיש קובץ PDF אשר מכיל את הפתרון היבש ב- gradescope תחת המשימה

"wet 1 dry part". **שימו לב שהחלק היבש חייב להיות מוקלד.**

- שימו לב כי אתם מגישים את כל שני החלקים הנ"ל, במטלות השונות.**
- לאחר שהגשתם, יש באפשרותכם לשנות את התוכנית ולהגיש שוב. ההגשה האחרונה היא הנחשבת.
- הערכת הציון שמופיעה ב-gradescope אינה ציונכם הסופי על המטלה. הציון הסופי יתפרסם רק לאחר ההגשות המאוחרות של משרתי המילואים.
- במידה ואתם חושבים שישנה תקלה מהותית במערכת הבדיקה ב-gradescope נא להעלות זאת בפורום הפיאצה ונטפל בה בהקדם.

דחיות ואיחורים בהגשה:

- דחיות בתרגיל הבית תינתנה אך ורק לפי תקנון הקורס.
- 5 נקודות יורדו על כל יום איחור בהגשה ללא אישור מראש. באפשרותכם להגיש תרגיל באיחור של עד 5 ימים ללא אישור. תרגיל שיוגש באיחור של יותר מ-5 ימים ללא אישור מראש יקבל 0.
- במקרה של איחור בהגשת התרגיל יש עדיין להגיש את התרגיל אלקטרונית דרך אתר הקורס.
- בקשות להגשה מאוחרת יש להפנות למתרגל האחראי בלבד בכתובת goldstein@campus.technion.ac.il. לאחר קבלת אישור במייל על הבקשה, מספר הימים שאושרו לכם נשמר אצלנו. לכן, אין צורך לצרף להגשת התרגיל אישורים נוספים או את שער ההגשה באיחור.

בהצלחה!